Sprint 11 · Visión por computador

¿podemos aprovechar las imágenes del cliente?

FECHAS
17–20 de noviembre

HORAS
17 h (5 h teoría · 7 h práctica · 5 h PBL) · **Módulo 4, UD 4.4–4.6**

Qué aprenderás

1. Cómo "ve" una red: imágenes como matrices, convolución, *pooling* y arquitectura de una CNN.
2. Entrenar una CNN pequeña y mejorar su generalización con *data augmentation*.
3. Reutilizar redes preentrenadas: *transfer learning* y *fine-tuning*.
4. Detección de objetos y segmentación: IoU, NMS, AP/mAP y un detector clásico con OpenCV.
5. Evaluar un clasificador de imágenes con métricas por clase y **análisis de errores**.

Requisitos previos

- Sprint 10 (redes neuronales, Keras, curvas de aprendizaje, regularización).
- Todo funciona **sin internet** con imágenes sintéticas (dibujos generados por código, no fotos reales).
- Algunas celdas opcionales para Colab (descarga de `tf_flowers`, pesos de MobileNetV2 con ImageNet, YOLO) están marcadas como **«requiere internet»**: en el taller no se ejecutan (llevan un interruptor `USAR_INTERNET = False`).

Prácticas

Notebook	Tema	Tiempo
`S11_01_cnn`	Convolución, pooling y CNN en Keras (dígitos 8×8); *data augmentation*	1 h 45
`S11_02_transfer_learning`	Transfer learning y fine-tuning (con preentrenamiento propio y MobileNetV2 en Colab)	1 h 45
`S11_03_deteccion_segmentacion`	IoU, NMS, mAP, detector con OpenCV, segmentación (y YOLO opcional)	1 h 45
`S11_04_reto_vision_cliente`	Esqueleto del reto: dataset por carpetas, transfer learning, análisis de errores	1 h 45 (+ trabajo PBL)

Material extra: `generar_dataset_sintetico.py` crea el mini dataset de tipos de alojamiento (una carpeta por clase) para probar el flujo. No se sube al repositorio: se genera con el script (`python generar_dataset_sintetico.py`).

RETO Reto PBL · «Visión artificial para el cliente»

Contexto del cliente. TurisData Canarias recibe cientos de fotos de alojamientos y zonas cada semana y las clasifica una persona a mano (hotel, apartamentos, casa rural, camping, piscina). Quiere saber si un modelo puede pre-clasificarlas y, sobre todo, **en qué se equivoca**, para decidir cuánto puede fiarse de él y qué fotos debe revisar una persona.

Qué se entrega y en qué formato

1. **Modelo** entrenado con *transfer learning* (archivo `.keras` o notebook que lo reproduce).
2. **Demostrador:** una función (o una pequeña interfaz en Colab) que recibe una foto y devuelve clase, confianza y «revisar a mano» si es baja.
3. **Análisis de errores** de 1–2 páginas (PDF o Markdown): métricas por clase, matriz de confusión y **al menos 6 ejemplos mal clasificados** con su hipótesis de causa.

Restricciones

- Dataset: el sintético que trae el notebook, o uno propio con **licencia clara** (fotos propias o con licencia abierta; guarda autor y licencia). Sin personas reconocibles.
- Estructura de carpetas: una por clase; mínimo 30 imágenes por clase (mejor 100 o más).
- Partición entrenamiento/validación/prueba fija; la prueba se usa una sola vez.
- Debe ejecutarse en CPU en pocos minutos; en Colab puedes usar MobileNetV2 con ImageNet (opcional).

Criterios de evaluación (0–4, la media alimenta «ejercicios y entregas»)

Criterio	Qué buscamos aquí
Corrección técnica	Carga y partición sin fugas, *transfer learning* bien aplicado (congelar, cabeza, *fine-tuning* con tasa baja), métricas por clase y comparación con una referencia (desde cero).
Reproducibilidad	Se ejecuta desde el README; semillas y versiones fijadas; script o instrucciones para obtener el dataset.
Análisis y comunicación al cliente	Errores analizados con ejemplos y causas; recomendación clara (qué se automatiza y qué se revisa a mano) con umbral de confianza justificado.
IA responsable	Riesgos concretos: representatividad del dataset, clases poco frecuentes, etiquetas dudosas, privacidad y licencias de las fotos; medidas aplicadas.

Pasos sugeridos

- **Día 1:** ejecuta `S11_04`, revisa la estructura del dataset (`contar_por_clase`) y decide si usarás el sintético o uno propio.
- **Día 2:** entrena el modelo con *transfer learning* y una referencia desde cero; mide en validación.
- **Día 3:** evalúa en prueba y haz el análisis de errores (matriz de confusión, ejemplos, hipótesis).
- **Día 4:** demostrador y umbral de confianza (cobertura frente a precisión).
- **Día 5:** informe y presentación de 5 minutos.

Qué se enseña en la *review*. El demostrador funcionando con 2–3 fotos, la matriz de confusión y tres errores comentados ("el modelo confunde X con Y porque… y propondríamos…").

Definition of Done

- [ ] El notebook se ejecuta de arriba abajo con semillas fijas.
- [ ] El dataset tiene estructura por carpetas y se ha comprobado su contenido por clase.
- [ ] Entrenamiento/validación/prueba separados; la prueba se usa una sola vez.
- [ ] Hay modelo con *transfer learning* y una referencia para compararlo.
- [ ] Métricas por clase y matriz de confusión interpretadas.
- [ ] Al menos 6 ejemplos mal clasificados analizados, con hipótesis y mejora propuesta.
- [ ] Demostrador con umbral de confianza y opción «revisar a mano».
- [ ] Se documentan origen y licencia de los datos y al menos dos riesgos de IA responsable.

Recursos

- Keras: guías «Transfer learning & fine-tuning» y «Image classification from scratch» (keras.io).
- Documentación de `keras.utils.image_dataset_from_directory`.
- Documentación de OpenCV para Python: `cv2.findContours`, `cv2.matchTemplate`.
- Guía de *Ultralytics YOLO* (docs.ultralytics.com), solo para la celda opcional de Colab.
- Creative Commons: cómo reconocer y citar licencias de imágenes (creativecommons.org).